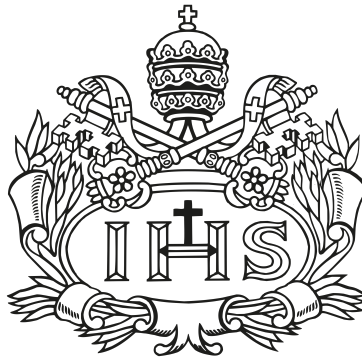


PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
2026



Pontificia Universidad
JAVERIANA
Colombia

Taller 3 Posix

Autores

María Alejandra Rodriguez

Sistemas Operativos

Profesor

John Corredor Franco

Fecha de entrega

19 de Mayo, 2026

1. Descripción del problema

En este taller el reto del sistema consiste en gestionar múltiples procesos ejecutándose concurrentemente, donde varios agentes de medición transmiten información simultáneamente hacia un proceso central de monitoreo. Esto implica resolver problemas asociados a:

- comunicación entre procesos (IPC),
- sincronización de acceso a recursos compartidos,
- concurrencia,
- control de buffers,
- exclusión mutua,
- y procesamiento seguro de la información.

El sistema debe garantizar que únicamente las mediciones válidas sean procesadas, descartando aquellas que no cumplan los rangos establecidos para los parámetros meteorológicos. Para solucionar esta problemática, se implementó una arquitectura concurrente basada en procesos independientes, comunicación mediante pipes FIFO, uso de hilos POSIX y sincronización utilizando semáforos y mutex bajo el modelo productor-consumidor. El sistema desarrollado permite consolidar las mediciones válidas y generar finalmente una clasificación meteorológica de acuerdo con las condiciones climáticas calculadas.

2. Funcionalidad de cada uno de los codigos

2.1. `common.h`

Funciona como una cabecera compartida entre los programas `producer.c`, `consumer.c` y `posixSincro.c`. Su propósito principal es centralizar las librerías, constantes y estructuras de datos utilizadas por los diferentes archivos del proyecto. Dentro de este archivo se incluyen las librerías necesarias para trabajar con semáforos POSIX, memoria compartida, hilos y funciones generales del sistema operativo Linux. Además, se define el tamaño del buffer compartido mediante constantes globales.

2.2. `consumer.c`

El programa implementa el proceso consumidor. Su función principal es leer y retirar los datos generados previamente por el productor desde el buffer compartido. El consumidor utiliza los mismos semáforos POSIX para sincronizar el acceso al buffer. Si no existen datos disponibles, el consumidor espera hasta que el productor agregue nuevos elementos. Una vez consume los datos, libera espacio en el buffer y al finalizar elimina los recursos compartidos utilizados, como la memoria compartida y los semáforos.

2.3. `producer.c`

Este código implementa el proceso productor dentro del modelo productor-consumidor. Su función principal es generar datos y almacenarlos en un buffer compartido utilizando memoria compartida POSIX. El productor utiliza semáforos con nombre para controlar el acceso al buffer. Antes de insertar un dato, verifica si existe espacio disponible; si el buffer está lleno, el proceso queda bloqueado hasta que el consumidor libere espacio. A medida que produce datos, los almacena en posiciones del buffer circular y notifica al consumidor que existen nuevos elementos disponibles para ser consumidos.

2.4. `posixSincro.c`

Implementa concurrencia mediante hilos POSIX (pthread) utilizando el modelo productor-consumidor dentro de un mismo proceso. El programa crea múltiples hilos productores encargados de generar mensajes y almacenarlos en un buffer compartido, mientras que un hilo consumidor denominado spooler se encarga de leer e imprimir dichos mensajes. Para evitar accesos simultáneos incorrectos al buffer, el programa utiliza mutex y variables de condición, garantizando sincronización y exclusión mutua entre los hilos concurrentes.

2.5. `Makefile`

El proyecto incluye un archivo Makefile cuya función principal es automatizar el proceso de compilación, ejecución y limpieza de los programas desarrollados en el taller. Mediante la regla all, el Makefile permite compilar automáticamente los ejecutables `producer`, `consumer` y `posixSincro` utilizando un único comando (`make all` o simplemente `make`), evitando escribir manualmente las instrucciones de compilación con `gcc`.

Adicionalmente, el Makefile incorpora reglas para ejecutar programas, eliminar ejecutables generados y limpiar recursos POSIX utilizados durante las pruebas, como memoria compartida y semáforos. Esto facilita la organización, reutilización y mantenimiento del proyecto, siguiendo prácticas comunes utilizadas en entornos Linux y desarrollo de sistemas concurrentes.

3. Actividad 1 – Productor–Consumidor con Semáforos POSIX

En la primera actividad se implementó el modelo clásico productor-consumidor utilizando procesos independientes sincronizados mediante semáforos POSIX y memoria compartida.

Para esta solución se desarrollaron los programas:

```
1 producer.c
2 consumer.c
```

y el archivo compartido:

```
1 posix.h
```

El programa `producer.c` se encarga de generar datos y almacenarlos dentro de un buffer circular compartido utilizando memoria compartida POSIX mediante:

```
1 shm_open()
2 mmap()
```

Por otra parte, `consumer.c` consume los datos almacenados por el productor desde la memoria compartida.

Para controlar el acceso concurrente al buffer se implementaron semáforos POSIX utilizando:

```
1 sem_open()
2 sem_wait()
3 sem_post()
```

Los semáforos permitieron sincronizar:

- espacios disponibles en el buffer,
- datos disponibles para consumir,
- y el acceso concurrente entre productor y consumidor.

Además, se implementó un buffer circular para reutilizar posiciones del arreglo compartido y optimizar el manejo de memoria.

Con esta actividad se aplicaron conceptos de:

- concurrencia,
- comunicación entre procesos (IPC),
- memoria compartida,
- sincronización,
- y exclusión mutua.

4. Actividad 2 – Hilos POSIX y Concurrencia

En la segunda actividad se implementaron soluciones concurrentes utilizando hilos POSIX (`pthread`) para demostrar procesamiento paralelo y sincronización entre hilos dentro de un mismo proceso.

La primera implementación fue:

```
1 posixSincro.c
```

Este programa aplica nuevamente el modelo productor-consumidor, pero utilizando múltiples hilos productores y un hilo consumidor (spooler). Los productores generan mensajes concurrentemente y los almacenan en un buffer compartido, mientras el spooler consume e imprime dichos mensajes.

Para sincronizar el acceso al buffer compartido se utilizaron:

- mutex POSIX,
- variables de condición,
- y buffers circulares.

Las principales funciones utilizadas fueron:

```
1 pthread_create()
2 pthread_join()
3 pthread_mutex_lock()
4 pthread_mutex_unlock()
5 pthread_cond_wait()
6 pthread_cond_signal()
```

Con esta implementación se evitó:

- condiciones de carrera,
- accesos simultáneos incorrectos,
- y desbordamientos del buffer.

Adicionalmente, como complemento de la actividad, se desarrolló:

```
1 concurrenciaPosix.c
```

Este programa implementa procesamiento paralelo utilizando múltiples hilos POSIX para buscar concurrentemente el valor máximo de un vector.

El vector se divide en diferentes segmentos y cada hilo procesa una parte distinta calculando un máximo parcial. Finalmente, el hilo principal reúne los resultados parciales y determina el máximo global.

Esta implementación permitió reforzar conceptos como:

- división de trabajo entre hilos,
- paralelismo de datos,
- procesamiento concurrente,
- creación y coordinación de hilos POSIX.

5. Conclusiones, observaciones y recomendaciones

5.1. Conclusiones

El desarrollo de ambas actividades permitió aplicar de manera práctica conceptos fundamentales de concurrencia y sincronización en sistemas operativos Linux. Se logró implementar correctamente el modelo productor-consumidor utilizando tanto procesos independientes como hilos POSIX. En la primera actividad se comprobó el funcionamiento de los semáforos

POSIX y la memoria compartida como mecanismos eficientes para la comunicación y sincronización entre procesos. En la segunda actividad se evidenció la importancia del uso de mutex y variables de condición para proteger recursos compartidos y evitar condiciones de carrera en aplicaciones multihilo. El uso de buffers circulares permitió gestionar de manera organizada el intercambio de información entre productores y consumidores. Las actividades permitieron comprender cómo funcionan internamente los mecanismos de sincronización y concurrencia implementados por los sistemas operativos modernos. Se fortalecieron habilidades relacionadas con programación concurrente, manejo de procesos, hilos y administración segura de recursos compartidos.

5.2. Recomendaciones

Implementar manejo adicional de errores para fortalecer la robustez del sistema. Utilizar herramientas de depuración y monitoreo de hilos para analizar el comportamiento concurrente del programa. Realizar pruebas con diferentes tamaños de buffer para evaluar el rendimiento del sistema. Implementar mecanismos adicionales de control para manejar posibles bloqueos o deadlocks. Continuar profundizando en técnicas de sincronización y comunicación entre procesos, ya que son fundamentales en sistemas distribuidos y aplicaciones concurrentes. Como mejora futura, se podrían implementar sockets o colas de mensajes para ampliar las capacidades de comunicación entre procesos.